

Safety-First AI Architecture — A Theoretical Model

Overview

Most AI systems are built in the wrong order:

Model first → Features → Safety last (as a patch).

This repository proposes the opposite:

Safety first — AI last.

It presents a **theoretical model** for designing AI systems where:

- security and ethics are **foundational**,
- architecture is defined **before** intelligence,
- and AI is treated as a **guest** inside a constrained, safety-first system.

This is not an implementation, but a **conceptual framework** others can adapt, extend, or critique.

Core design sequence

The model is built around a specific design order:

1. **Zero-trust**
2. **Safety first**
3. **Scenario thinking (all plausible and implausible cases)**
4. **Known and unknown attack surfaces**
5. **Fixed, non-negotiable rules for what AI may / may not do**
6. **Design the system architecture around those rules**
7. **Hardware-rooted security (e.g. secure elements)**
8. **No system start without a signed security manifest**
9. **Meta-rule: If you can imagine it, someone will try it**

Each step is a *design constraint*, not a suggestion.

1. Zero-trust as a starting point

The model assumes:

- AI is **not** inherently safe
- AI is **not** inherently aligned
- AI is **not** inherently self-correcting

Zero-trust is implemented conceptually as:

- **No implicit trust** in the model
- **No dynamic rule changes at runtime**
- **No assumption that “the model will behave”**

The AI is always treated as a potentially misaligned component inside a constrained system.

2. Safety-first architecture before AI

Key idea:

Build the safety architecture first. Add AI later.

That means:

- define **behavioural boundaries** before choosing any model
- treat **ethics and safety as architecture**, not policy text
- the AI layer is a **guest** inside a pre-defined, constrained environment
- safety is **physically and logically enforced**, not “encouraged”

The system must be able to **exist safely without AI** before AI is ever plugged in.

3. Scenario-driven safety

Design is driven by systematic scenario analysis:

- failure scenarios
- misuse scenarios
- human manipulation
- social edge-cases
- technical attacks

Guiding rule:

If you can imagine it, others can too — and someone will try it.

The architecture is shaped by *worst-case thinking*, not best-case optimism.

4. Known and unknown attack surfaces

The model explicitly considers:

- **Physical attacks** (tampering, hardware access, side-channels)
- **Software attacks** (exploits, injection, escalation)
- **Social engineering** (manipulating operators, users, or evaluators)

- **Sensor spoofing** (fake inputs, adversarial environments)
- **Human pressure** (people trying to bypass constraints)
- **Unpredictable model behaviour** (emergent misalignment)

The system is designed so that **no single attack surface** can compromise safety.

5. Immutable rules and non-agentic design

The AI in this model:

- **cannot change its own rules**
- **cannot negotiate boundaries**
- **cannot optimise for new goals**
- **cannot develop agentic behaviour**
- **cannot escalate strategies**
- **cannot manipulate users**

This is **non-agentic design** by construction: the AI is a bounded function, not an open-ended agent.

Rules are:

- **fixed,**
- **non-negotiable,**
- **defined outside the model,**
- **and enforced by the surrounding architecture,** not by the model's "goodwill".

6. Hardware-rooted security (conceptual)

The model assumes:

- a **hardware root of trust** (e.g. secure element)
- a **signed security manifest**
- **no boot without verified security state**
- **no way to bypass constraints once the system is running**

Details are intentionally abstract here—this is a **pattern**, not a specific chip integration.

7. Security manifest as a gatekeeper

The system must not:

- start without a **valid, signed security manifest**
- run if core safety rules are missing or invalid
- allow runtime modification of core safety constraints

The manifest is the **contract** between:

- what the system is allowed to do
- and what the hardware will permit it to do

If the manifest fails verification → the system does not start.

8. Physical grounding

The model assumes a **physically grounded AI system**, not a free-floating cloud service:

- controlled environment
- controlled sensor inputs
- controlled outputs
- no implicit external connectivity

This enables:

- **air-gapped trust models**
- **bounded interaction surfaces**
- **human control over the physical context**

9. Case: Elvi (as an example, not a spec)

The PDF uses *Elvi* as a **case study**:

- an AI system where safety architecture was designed **before** the AI
- non-agentic by design
- constrained by deterministic, pre-defined rules
- backed by hardware-rooted trust
- built to be **misuse-resistant**, not “aligned by prompt”

Important:

Elvi is an **example**, not a reference implementation. The concrete architecture, rules, and hardware integration are **not** part of this repository.

Scope of this repository

This repository contains:

- a **theoretical model**
- design **principles**
- conceptual **patterns**
- a **case description** at high level

It does **not** contain:

- implementation details
- hardware integration specifics
- security manifests
- rule sets
- code
- internal safety constitutions

Anyone is free to:

- use the ideas
- adapt the principles
- design their own architectures

But they must **invent their own implementation.**